

02. WSL 側実行ルータ

solar_ws0129-main

2026-07-29

Contents

1 対象	2
2 このファイルを読む理由	2
3 呼び出し関係	2
3.1 どこから呼ばれるか	2
3.2 次に読むと理解が進むファイル	2
3.3 同一リポジトリ内の主要依存	2
4 ソース冒頭の把握	2
4.1 代表コード断片	2
5 主要構造の読み取り	4
5.1 ファイルを上から読んだときの定義順	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 CLI、PowerShell、Bash、環境変数、終了コード	5
6.3 例外、try/finally、with、資源解放	6
6.4 天候、route、補間、時刻、単位	6
6.5 launch Action、Node Action、実行可能名、remapping	6
6.6 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか	6
7 関数・クラスを上から順に解説	7
7.1 shell 分岐と外部コマンド	7
8 処理の流れ	8
9 このファイルの位置づけ	9

1 対象

項目	内容
ファイル	scripts/solar_control.sh
ソース SHA-256	3a3beda8fd115e6a5f5ce0974d68273fb14f341f19bd1ad72a72fcad44d5ebca
種別	Bash
区分	入口
役割	ROS 2 build、launch、offline simulation、forecast、identification、learn を mode 別に振り分ける。
起動文脈	SolarSim.ps1 から WSL 内で呼ばれる実行ルータ。

2 このファイルを読む理由

ROS 2 build、launch、offline simulation、forecast、identification、learn を mode 別に振り分ける。

- mode から launch file を決める。
- ROS 環境を source する。
- 起動 PID 管理と stop/status/graph を一括で持つ。

3 呼び出し関係

3.1 どこから呼ばれるか

- SolarSim.ps1

3.2 次に読むと理解が進むファイル

- launch/solarcar_sim.launch.py
- launch/solar_race_live.launch.py
- scripts/solar_sim.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
```

```

RUN_DIR="${ROOT_DIR}/.run"
DISTRO_NAME="${ROS_DISTRO:-humble}"

mkdir -p "${RUN_DIR}"
export AMENT_TRACE_SETUP_FILES="${AMENT_TRACE_SETUP_FILES-}"

action="${1:-up}"
mode="${2:-sim}"
profile="${3:-config/solar/bwsc_2027_demo.yaml}"

case "${mode}" in
    sim)
        launch_file="solarcar_sim.launch.py"
        ;;
    measure)
        launch_file="solar_measurement.launch.py"
        ;;
    live)
        launch_file="solar_race_live.launch.py"
        ;;
    live_wifi)
        launch_file="solar_race_live_wifi.launch.py"
        ;;
    *)
        echo "unknown mode: ${mode}" >&2
        exit 2
        ;;
esac

PID_FILE="${RUN_DIR}/solar_${mode}.pid"
LOG_FILE="${RUN_DIR}/solar_${mode}.log"
GRAPH_BASE="${ROOT_DIR}/rqt_graph_solar_${mode}"

source_ros() {
    set +u
    # shellcheck disable=SC1091
    source "/opt/ros/${DISTRO_NAME}/setup.bash"
    if [[ -f "${ROOT_DIR}/install/setup.bash" ]]; then
        # shellcheck disable=SC1091
        source "${ROOT_DIR}/install/setup.bash"
    fi
    set -u
}

is_running() {
    [[ -f "${PID_FILE}" ]] || return 1
    local pid
    pid="$(cat "${PID_FILE}")"
    [[ -n "${pid}" ]] || return 1
    ps -p "${pid}" -o pid= 2>/dev/null | grep -q .
}

find_matching_pids() {
    ps -eo pid=,args= | awk '
        $0 ~ /\opt\/ros\/humble\/bin\/ros2 launch mpc_solarcar solar(car_sim|_measurement|
        _race_live|_race_live_wifi)\.launch\.py/ || $0 ~ /install\/mpc_solarcar\/lib\/
        mpc_solarcar\/(gps_sim_node|mpc_node|solar_state_node|dashboard_node|solar_logger_node|
        distance_node|grade_node|weather_fetch_node|solar_autocal_node|speed_command_bridge_node|
        telemetry_text_bridge_node|wind_correction_node)/ {
            print $1
        }
    '
}

```

```

}

has_matching_processes() {
    [[ -n "$(find_matching_pids)" ]]
}

kill_matching_processes() {
    local pid
    while read -r pid; do
        [[ -n "${pid}" ]] || continue
        kill -TERM "${pid}" 2>/dev/null || true
    done < <(find_matching_pids)
}

stop_launch() {
    if is_running; then
        local pid
        pid="$(cat "${PID_FILE}")"

```

5 主要構造の読み取り

action 分岐は 20 件。

5.1 ファイルを上から読んだときの定義順

行	内容
2	set -euo pipefail
4	ROOT_DIR="\$(cd "\$(dirname "\${BASH_SOURCE[0]}")/.." && pwd)"
5	RUN_DIR="\${ROOT_DIR}/.run"
6	DISTRO_NAME="\${ROS_DISTRO:-humble}"
8	mkdir -p "\${RUN_DIR}"
9	export AMENT_TRACE_SETUP_FILES="\${AMENT_TRACE_SETUP_FILES-}"
11	action="\${1:-up}"
12	mode="\${2:-sim}"
13	profile="\${3:-config/solar/bwsc_2027_demo.yaml}"
15	case "\${mode}" in
16	sim)
17	launch_file="solarcar_sim.launch.py"
18	;;
19	measure)
20	launch_file="solar_measurement.launch.py"
21	;;
22	live)
23	launch_file="solar_race_live.launch.py"
24	;;
25	live_wifi)
26	launch_file="solar_race_live_wifi.launch.py"
27	;;
28	*)
29	echo "unknown mode: \${mode}" >&2

```

30     exit 2
31 ;;
32 esac
34 PID_FILE="${RUN_DIR}/solar_${mode}.pid"
35 LOG_FILE="${RUN_DIR}/solar_${mode}.log"
36 GRAPH_BASE="${ROOT_DIR}/rqt_graph_solar_${mode}"
38 source_ros() {
39     set +u
41     source "/opt/ros/${DISTRO_NAME}/setup.bash"
42     if [[ -f "${ROOT_DIR}/install/setup.bash" ]]; then
44         source "${ROOT_DIR}/install/setup.bash"
45     fi
46     set -u
47 }
49 is_running() {
50     [[ -f "${PID_FILE}" ]] || return 1

```

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```

node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。

```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 CLI、PowerShell、Bash、環境変数、終了コード

CLI は端末からプログラム名と引数を渡す操作界面である。‘argparse’は文字列として届く引数を名前、型、既定値、必須性に従って解析する。

PowerShell と Bash は別の shell であり、変数記法、改行継続、引用、パス表記が異なる。このプロジェクトでは Windows 側の SolarSim.ps1 が WSL 側の solar_control.sh へ処理を渡す。

環境変数は親プロセスから子プロセスへ受け渡される名前付き文字列である。ROS_DOMAIN_ID、RMW_IMPLEMENTATION、Python の数値スレッド数などはコード外から動作を変えるため、実行記録へ残す必要がある。

終了コード 0 は一般に成功、0 以外は失敗を示す。shell ルータは子プロセスの終了コードを握り潰さず上位へ返すことで、自動運用が失敗を検知できる。

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)

6.3 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.4 天候、route、補間、時刻、単位

予報は時刻だけの系列か、時刻と route 距離の 2 次元 grid になり得る。現在時刻 t と距離 s が grid 点の間なら、周囲の値から補間して日射、温度、風を求める。

UTC、現地 timezone、naive datetime を混ぜると数時間のずれが発生する。入力で timezone を確定し、内部比較は UTC、表示は現地時刻という役割分担が安全である。

route 距離の km、速度の km/h と m/s、時間の秒、energy の Wh と J を跨ぐため、変換係数 3.6、3600、1000 の意味を各式で確認する。

6.5 launch Action、Node Action、実行可能名、remapping

‘launch_ros.actions.Node(...)’は rclpy の Node 基底クラスではなく、指定した package の executable をプロセスとして起動する launch Action である。

‘DeclareLaunchArgument’は launch 実行時に受け取る入力欄を宣言し、‘LaunchConfiguration’はその値を後で解決する substitution を表す。‘perform(context)’は実行時 context から確定文字列を取り出す。

launch の ‘name’は Node 名 override、‘parameters’は起動時 parameter、‘remappings’は Node や topic の既定名を別名へ対応付ける。launch は Python ファイルから main という名前を推測せず、executable としてインストールされた console script を起動する。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [setuptools 公式: Entry Points / Console Scripts](#)

6.6 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```

ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph

```

rosvbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```

ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_kmh /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock

```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

7 関数・クラスを上から順に解説

このファイルでは独立した関数・クラス定義が少ない。

7.1 shell 分岐と外部コマンド

行	種別	内容
16	action	sim
19	action	measure
22	action	live
25	action	live_wifi
222	action	build
225	action	start
228	action	stop
231	action	restart
235	action	status
238	action	graph
241	action	up
246	action	simulate
250	action	historical-weather
254	action	historical-simulate

258	action	forecast
262	action	identify
266	action	fit
270	action	learn
274	action	audit
277	action	log
59	command	<code>\$0~/\opt\ros\humble\bin\ros2launchmpc_solarcarsolar(car_sim _measurement _race_live _race_live_wifi)\.launch.py/ \$0~/install/mpc_solarcar\lib\mpc_solarcar\gps_sim_node mpc_node solar_state_node dashboard_node solar_logger_node distance_node grade_node weather_fetch_node solar_autocal_node speed_command_bridge_node telemetry_text_bridge_node wind_correction_node)/{</code>
108	command	<code>colconbuild--packages-selectmpc_solarcar</code>
122	command	<code>launch_cmd="exportAMENT_TRACE_SETUP_FILES=\\${AMENT_TRACE_SETUP_FILES-};cd\${root_q}&&set+u&&source/opt/ros/\${DISTRO_NAME}/setup.bash&&source\${root_q}/install/setup.bash&&set-u&&execros2launchmpc_solarcar\${launch_file}profile_yaml:=\${profile_q}"</code>
123	command	<code>nohupsetsidbash-lc"\${launch_cmd}">>"\${LOG_FILE}"2>&1</dev/null&</code>
128	command	<code>echo"launchfailed:ros2launchexitedimmediately">&2</code>
152	command	<code>python3"\${ROOT_DIR}/scripts/export_rqt_graph.py"--output-base"\${GRAPH_BASE}"--wait-sec20--mode"\${mode}"</code>
158	command	<code>python3"\${ROOT_DIR}/scripts/solar_sim.py"--profile_yaml"\${profile}"</code>
164	command	<code>python3"\${ROOT_DIR}/scripts/build_historical_weather_counterfactual_grid.py"\</code>
173	command	<code>python3"\${ROOT_DIR}/scripts/solar_sim.py"--profile_yaml"\${historical_profile}"</code>
179	command	<code>python3"\${ROOT_DIR}/scripts/fetch_weather_forecast.py"--profile_yaml"\${profile}"</code>
185	command	<code>python3"\${ROOT_DIR}/scripts/run_identification_pipeline.py"--profile_yaml"\${profile}"</code>
196	command	<code>python3"\${ROOT_DIR}/scripts/run_vehicle_identification.py"\</code>
204	command	<code>python3"\${ROOT_DIR}/scripts/tune_upper_planner_weights.py"\</code>
210	command	<code>python3"\${ROOT_DIR}/scripts/generate_package_inventory.py"</code>
211	command	<code>python3"\${ROOT_DIR}/scripts/audit_solar_package.py"</code>
212	command	<code>python3-mpytest-q</code>

8 処理の流れ

1. action と mode を受け取る。
2. ROS 環境と install/setup.bash を読む。

3. launch 実行か offline script 実行かを分岐する。
4. 必要に応じて PID 管理、graph 出力、log tail を行う。

9 このファイルの位置づけ

この資料群では、`scripts/solar_control.sh` を **入口**の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。